

Dalhousie MCS

Pre-Master's Learning & Research Preparation Guide

A 12-week academic reboot for a software engineer entering a thesis-based Master of Computer Science

Purpose

This guide is designed to rebuild academic fluency rather than teach programming from scratch. The target is to arrive at Dalhousie ready to read research papers, reason mathematically, design experiments, evaluate evidence, and begin thesis work efficiently.

Core principle: study → implement → experiment → write → question.

Recommended workload: 10–15 hours/week for 12 weeks. If time is limited, preserve the research, statistics, algorithms, and writing components.

Program context: thesis-based Master of Computer Science (MCS), not the Master of Applied Computer Science (MACS).

1. How to Use This Guide

You already have professional software-engineering experience. Do not treat this as a beginner curriculum. Skip syntax tutorials you already know and spend your time rebuilding the skills that professional work often exercises less frequently: proofs, statistics, research methodology, paper criticism, controlled experiments, and academic writing.

Weekly rhythm

Day	Primary activity	Output
Mon	Theory / lecture	Notes + 3 questions
Tue	Theory + problems	Solved exercises
Wed	Implementation	Working code
Thu	Experiment	Data + measurements
Fri	Research paper	1-page paper note
Sat	Project / analysis	Plots + interpretation
Sun	Review	Weekly summary + next plan

The most important rule

Never let a course become passive video consumption. Every major topic should end with a concrete artifact: code, a solved problem set, an experiment, a plot, a research note, or a short written argument.

Definition of “ready”

- You can analyze an algorithm and explain why its complexity is what it is.
- You can read a research paper and identify the problem, contribution, method, baseline, metrics, and threats to validity.
- You can run a reproducible Python experiment and report uncertainty rather than a single lucky number.
- You can write a concise methodology and results section.
- You can use Git/Linux/LaTeX comfortably enough that tooling does not distract from research.

2. The 12-Week Roadmap

Week	Main focus	Topics	Required output
1	Algorithms reboot	Big-O, arrays, stacks, queues, recursion	Complexity notes + implementations
2	Data structures	Trees, heaps, hashing, graphs	Benchmark notebook
3	Algorithmic thinking	Sorting, BFS/DFS, shortest path, DP, greedy	5-8 solved problems + analysis
4	Discrete mathematics	Logic, sets, proofs, induction, graphs, counting	Proof/problem set
5	Probability	Random variables, expectation, variance, distributions	Simulation notebook
6	Statistics	Sampling, CI, hypothesis tests, effect size, regression	Statistical experiment
7	ML foundations	Regression, trees, SVM, clustering, evaluation	ML baseline project
8	Experimental ML	CV, leakage, tuning, error analysis, reproducibility	Model comparison report
9	Software research	Repository mining, defects, code quality, testing	GitHub mining study
10	Research reading	Methodology, baselines, validity, reproducibility	3 paper critiques
11	Paper reproduction	Reimplement a manageable published method	Reproduction report
12	Mini research project	Question → method → experiment → results	4-6 page mini-paper

Suggested weekly effort

6 h theory + 4 h implementation/experiments + 2 h paper reading + 1-3 h writing/review. If you have only 8 hours, reduce theory before reducing experiments and writing.

3. Course Track — Algorithms & Data Structures

Primary source

MIT 6.006 — [Introduction to Algorithms](#) — lectures, notes, problem sets and programming assignments. The course emphasizes algorithms, data structures, mathematical modeling, and performance analysis. `[cite]turn0search8`

Study these topics

- Asymptotic notation: O , Ω , Θ ; best/average/worst-case reasoning
- Arrays, linked lists, stacks, queues, hash tables
- Binary search trees, balanced trees, heaps and priority queues
- Sorting: insertion, merge, quicksort, heap sort; lower bounds
- Graph representations, BFS, DFS, topological sorting
- Shortest paths: Dijkstra and Bellman–Ford at a conceptual/implementation level
- Greedy algorithms and when greedy choice is justified
- Dynamic programming: state definition, recurrence, memoization, reconstruction
- Amortized analysis at a basic level

Experiments

- 1 Implement four sorting algorithms and benchmark them over input sizes from 10^2 to 10^5 .
- 2 Compare list search, binary search, and hash lookup.
- 3 Generate random graphs and compare BFS/DFS runtime as graph density changes.
- 4 Implement a dynamic-programming solution and compare it with a naive recursive version.

Deliverable

One notebook containing runtime tables, plots, experimental conditions, and a short interpretation. Do not merely paste benchmark numbers—explain why the curves look the way they do.

4. Course Track — Discrete Mathematics & Mathematical Thinking

Primary source

[MIT Mathematics for Computer Science](#) — a current MIT course covering proofs, sets, relations, graph theory, state machines, induction, recurrences, asymptotics, counting, probability and related topics.

[\[cite\turn0search3\]](#)

The 2024 course provides lecture notes and problem sets; its readings use the freely available Mathematics for Computer Science text. [\[cite\turn0search6\]](#)

Topics to refresh

- Propositional and predicate logic
- Sets, functions, relations and equivalence relations
- Direct proof, contradiction, contrapositive and induction
- Graph theory: paths, connectivity, trees, directed graphs
- Combinatorics: permutations, combinations, counting arguments
- Recurrences and asymptotic reasoning
- Discrete probability and expectation

Practice rule

For each concept, solve problems without looking at the solution first. The objective is not memorization; it is becoming comfortable constructing an argument.

5. Course Track — Probability & Statistics

Why this matters

A thesis is often an evidence-generation process. You need enough statistics to distinguish a real effect from noise, avoid misleading comparisons, and communicate uncertainty.

Topics

- Probability rules and conditional probability
- Bayes' theorem
- Random variables, expectation and variance
- Common distributions: Bernoulli, binomial, normal, Poisson
- Sampling and the central limit theorem
- Confidence intervals
- Hypothesis testing and p-values
- Effect size and practical significance
- Correlation vs causation
- Linear regression and residuals
- Multiple comparisons and common statistical pitfalls

Mini-experiment

Simulate 10,000 repeated experiments in Python. Compare the distribution of sample means for $n=10$, 30, 100. Then construct confidence intervals and observe how coverage changes. Write one page explaining what you observed.

Suggested reference

Use MIT Mathematics for Computer Science for the discrete-probability foundation, then use An Introduction to Statistical Learning for applied statistical learning concepts. Prefer the current official edition/resource site when working through ML/statistics material.

6. Course Track — Machine Learning

Primary practical source

[Hands-On Machine Learning — Aurélien Géron](#) — the current O'Reilly edition covers the ML landscape, training/validation, model selection, overfitting, and practical ML workflows.

`[cite]turn0search7[`

Also useful

[Hands-On Machine Learning, 3rd edition](#) — the 2022 edition uses Scikit-Learn, Keras and TensorFlow and remains a useful practical reference. `[cite]turn0search11[`

Topics

- Linear and logistic regression
- Decision trees and random forests
- k-nearest neighbors and SVM
- k-means and PCA
- Train/validation/test split and cross-validation
- Data leakage and preprocessing pipelines
- Hyperparameter tuning
- Metrics: precision, recall, F1, ROC-AUC, MAE, RMSE
- Error analysis and class imbalance
- Reproducibility: random seeds, environment capture and experiment tracking

Do not over-focus on deep learning yet

Learn the conceptual chain—forward pass → loss → backpropagation → gradient descent → optimization—and run a small neural-network experiment. Save the deeper material for the direction your thesis eventually takes.

7. Research Skills — The Highest-Value Preparation

Paper-reading protocol

- 1 Pass 1 (10–15 min): title, abstract, introduction, figures, conclusion. State the problem and claimed contribution in your own words.
- 2 Pass 2: method, dataset, baselines, metrics, experimental setup and main results.
- 3 Pass 3: challenge assumptions. Ask whether the experiment actually supports the claims.
- 4 After reading: write three strengths, three weaknesses, one missing experiment, and one follow-up question.

Research note template

Paper: ...
Problem: ...
Research question / hypothesis: ...
Contribution: ...
Method: ...
Dataset / participants: ...
Baseline: ...
Metrics: ...
Main result: ...
Threats to validity: ...
What I would test next: ...

Research mindset

Replace “Did it work?” with “Compared with what, measured how, under which conditions, with what uncertainty, and does the evidence support the claim?”

8. Experimental Warm-Up Portfolio

Experiment	Question	Core skills	Output
1. Sorting benchmark	How do algorithms scale?	Algorithms, benchmarking	Plot + analysis
2. Search benchmark	When does $O(1)$ actually help?	Data structures, systems thinking	Runtime/memory report
3. ML model comparison	Which baseline performs best and why?	ML, metrics	Model comparison
4. Statistical simulation	How stable are estimates?	Probability, statistics	Simulation notebook
5. Repository mining	Do repository metrics relate to defects?	Git, APIs/data, stats	Dataset + analysis
6. Paper reproduction	Can I reproduce a published result?	Research methodology	Reproduction report

Experiment discipline

- State the research question before collecting results.
- Define independent/dependent variables and controls where applicable.
- Use repeated measurements when randomness or system noise is involved.
- Record environment, versions, hardware, random seeds and parameters.
- Keep raw data separate from processed data.
- Automate the experiment so you can rerun it.
- Interpret results conservatively; distinguish observation from explanation.

9. Recommended Capstone Warm-Up Project

Given your software-engineering background, I recommend a small Empirical Software Engineering + ML project.

Example research question

“Can repository-level and file-level historical metrics help predict which files are more likely to be associated with future defects?”

Possible pipeline

Git repositories → commit history → file metrics → issue/bug labels → dataset → baseline ML models → statistical evaluation → error analysis → report

Possible features

- Lines changed
- Number of previous commits
- Number of contributors
- File age
- Change frequency
- Historical bug count
- Cyclomatic complexity or other code-quality metrics

Do not try to build a production-grade system. The objective is to practice the research loop: question → operationalization → dataset → baseline → experiment → analysis → limitation → conclusion.

10. Tooling to Set Up Before the MCS

Tool	Use it for	Target skill
Python	Experiments, analysis, ML	Clean environments + reproducible scripts
Jupyter	Exploration	Readable notebooks, not hidden state
NumPy / Pandas / SciPy	Numerical/statistical work	Vectorized analysis
Matplotlib	Plots	Clear scientific figures
scikit-learn	ML baselines	Pipelines + evaluation
Git + GitHub	Research code	Branches, history, reproducibility
Linux shell	Data/code workflows	grep/find/awk/sed/ssh
Docker	Environment reproducibility	Rebuildable execution
LaTeX / BibTeX	Academic writing	Citations, figures, references
VS Code or your preferred IDE	Development	Fast navigation/debugging

Suggested repository structure

```
mcs-preparation/  
■■■ algorithms/  
■■■ statistics/  
■■■ ml/  
■■■ experiments/  
■■■ papers/  
■■■ research-notes/  
■■■ reports/  
■■■ README.md
```

11. Academic Writing Practice

Every two weeks, turn one experiment into a short report. Use this structure:

Section	What to answer
Abstract	What did you investigate and what did you find?
Introduction	Why does the problem matter?
Research question	What exactly are you testing?
Methodology	How did you investigate it?
Experimental setup	What data, environment, baselines and parameters were used?
Results	What happened? Use tables/figures.
Discussion	What does it mean? What alternative explanations exist?
Threats to validity	Why might the conclusion not generalize?
Conclusion	What can you confidently claim?

Writing standard

Avoid claims stronger than the evidence. “Our experiment observed...” is often better than “Our method proves...”. Learn to separate result, interpretation, and speculation.

12. Source Library

Core sources

[MIT 6.006 — Introduction to Algorithms](#)

Algorithms, data structures, analysis and problem sets.

[MIT Mathematics for Computer Science](#)

Proofs, discrete math, graphs, asymptotics and probability.

[OSTEP — Operating Systems: Three Easy Pieces](#)

Operating-system foundations and systems thinking.

[An Introduction to Statistical Learning](#)

Statistics and machine-learning foundations with practical orientation.

[Hands-On Machine Learning — Géron](#)

Practical ML workflow, validation and model selection.

[ACM Digital Library](#)

Primary source for computer-science research papers.

[IEEE Xplore](#)

Research papers, conferences and journals.

[arXiv](#)

Preprints and recent research; verify important claims against peer-reviewed versions when possible.

[Overleaf](#)

LaTeX and collaborative academic writing.

How to use sources efficiently

- Pick one primary source per subject instead of collecting ten courses.
- Use textbooks as references; do not force yourself to read every chapter.
- Use official course problem sets to test understanding.
- For research, prioritize original papers over summaries and blog posts.
- Keep a bibliography from day one using BibTeX or a reference manager.

13. Pre-Master's Readiness Checklist

Area	I can...	Done
Algorithms	Explain Big-O and implement common data structures/algorithms	☐
Math	Read and construct basic proofs; work with graphs/counting	☐
Probability	Reason about conditional probability and random variables	☐
Statistics	Interpret confidence intervals, p-values and effect sizes	☐
ML	Build and evaluate baseline models without leakage	☐
Research	Critique methodology, baselines and threats to validity	☐
Experiments	Design reproducible experiments and record conditions	☐
Python	Create clean scripts/notebooks for data and experiments	☐
Git/Linux	Work comfortably from the command line and version control	☐
Writing	Produce a 4-6 page research-style report	☐
LaTeX	Create citations, figures, tables and references	☐

Final test

If you can choose a modest research question, find relevant papers, define a baseline, build an experiment, analyze the results statistically, and write a defensible conclusion, you are in a very good position to begin the MCS.

14. The Mindset to Carry Into Dalhousie

Your professional background means you already know how to build software. The master's asks you to become better at answering questions whose answers are not already known.

The transition looks like this:

Software engineering: “How can I build this reliably?”

Research: “What do we actually know about this problem, what do we not know, and what experiment could produce trustworthy evidence?”

Do not optimize for the number of courses completed before arrival. Optimize for the number of times you have completed the research loop:

Question → Literature → Hypothesis → Method → Experiment → Evidence → Analysis → Limitation → Conclusion → Next Question

A strong preparation period should leave you with a small portfolio of experiments and written reports—not just a list of videos watched.

Suggested next step

Start Week 1 with algorithms and simultaneously create the Git repository that will hold all your preparation work. By Week 12, that repository should contain enough evidence of your academic reboot to serve as a personal research notebook.

Prepared as a practical warm-up guide for Sudarshan’s thesis-based Dalhousie MCS preparation.